

TianShanOS

Complete Guide to the Security Page

Operations, verification, and recovery based on the current main branch

Account security · SSH keys · Host fingerprints · HTTPS / mTLS · Config Pack

Generated: 2026-09-01

Contents

1. Read this first	4
1.1 What the page can do	4
1.2 Do not confuse the roles	4
1.3 Connection requirements	5
1.4 Terms in plain language	5
2. Quick start for individual users	5
2.1 Replace default passwords immediately	5
Change passwords from Account Security	5
2.2 Create a key for SSH	6
2.3 Deploy the key to a server	6
2.4 Verify the connection correctly	7
2.5 Revoke access in the correct order	7
3. SSH keys and server management	8
3.1 Key lifecycle	8
3.2 Public and private key export	8
Export a public key	8
Export a private key	8
3.3 Deployed Hosts is not the remote source of truth	8
3.4 SSH host fingerprints	9
First connection	9
Fingerprint changed	9
Fingerprint storage risk	9
3.5 Import and export SSH host configurations	9
Export a host configuration	10
Import a host configuration	10
4. HTTPS and mTLS	10
4.1 Three identities to keep separate	10
4.2 Plan names and certificate purposes	10
4.3 Generate the HTTPS key	11
4.4 Generate and issue a CSR	11
4.5 Install the device certificate	11

- 4.6 Install the client CA chain and test mTLS 11
 - 4.7 Delete all PKI credentials 12
- 5. Config Pack 12**
 - 5.1 Trust model 12
 - 5.2 Current capability table 12
 - 5.3 Export the device certificate 13
 - 5.4 Verify a received package 13
 - 5.5 Do not use generic Apply as proof of configuration 13
 - 5.6 Developer export 14
- 6. Security incidents and recovery 14**
 - 6.1 Suspected SSH private-key exposure 14
 - 6.2 Server rebuild or fingerprint change 14
 - 6.3 Certificate expiry or HTTPS private-key exposure 14
 - 6.4 Remote authorization and local record disagree 14
 - 6.5 Confirm a recovery path before deletion 15
- 7. Troubleshooting 15**
- 8. Security limitations you must account for 16**

This guide is for individual users, security administrators, and engineers who operate TianshanOS devices. It explains what the current Security page actually does, what it does not do, and how to verify every important change.

Important: the full WebUI currently runs over HTTP. Do not enter a login password or SSH password, or export a private key, while connected through public Wi-Fi, a guest network, or any other untrusted network. Use an isolated management network and restrict access to port 80 on the device.

1. Read this first

1.1 What the page can do

The Security page contains six sections:

Section	Purpose	Typical individual use
Account Security	Let root change root/admin passwords or reset admin	Needed during initial setup
Key Management	Create SSH keys, view public keys, conditionally export private keys, deploy, revoke, and delete	Common
Deployed Hosts	View locally recorded servers, test, export, revoke, or remove records	Common
Known Host Fingerprints	View or delete SSH server identity records	Needed after rebuilds, replacements, or security warnings
HTTPS Certificate	Manage the device certificate, CSR, and client CA for the separate port 443 service	Advanced/enterprise
Config Pack	Encrypt, sign, and distribute <code>.tscfg</code> packages	Do not use the current generic flow to apply production configuration

1.2 Do not confuse the roles

Identity	Current meaning
admin	The frontend router permits access to Security and displays key, SSH, certificate, and Config Pack controls, but not Account Security.
root	The frontend displays the same Security controls plus Account Security, where root can set root/admin passwords or reset admin.
Developer device	A property of the device certificate, not a login role. It controls whether the device can export Config Packs and SSH host configurations. Signing in as root does not turn a normal device into a Developer device.

This table describes browser visibility, not a backend security boundary. In the generic `/api/v1/*` path, the write-operation login check is commented out, and endpoint `requires_auth` and `permission` metadata is not enforced when an API is called. Except for handlers such as password management that validate their own token, a page login does not prove that the underlying operation is authenticated or authorized. Never expose the device directly to an untrusted network.

1.3 Connection requirements

Before making a security change, confirm that:

- 1 Your computer and TianshanOS are on a management network you control.
- 2 The device IP in the browser belongs to the device you intend to manage.
- 3 A server administrator has confirmed the SSH address, port, and username.
- 4 The target account currently permits password authentication when you deploy or revoke an SSH key. Whether it remains enabled afterward is a server-policy decision; this guide does not recommend enabling it permanently as a troubleshooting shortcut.
- 5 You have another working administrative path before deleting, overwriting, or replacing credentials.

1.4 Terms in plain language

- NVS is the area of device flash used to store settings and key data. NVS Encryption is not enabled in the current build, so stored private keys must not be described as encrypted at rest.
- TOFU means trust on first use: accept the server identity seen on the first connection, then detect later changes. The current WebUI automatically performs this first trust decision.
- A CSR is a certificate request created with the device private key and sent to a CA. It does not contain the private key.
- A SAN is the DNS name or IP address that a certificate permits clients to use. Modern clients normally validate the address against SAN.
- A CA issues or validates certificates. A client CA installed on the device is not the same as the server trust store in your browser or computer.
- mTLS requires both sides to present certificates: the server certificate identifies the device and the client certificate identifies the connecting client.
- PEM is the text format beginning with -----BEGIN ...----- used for certificates, CSRs, and keys.
- EKU states a certificate purpose, such as server authentication or client authentication.
- A Config Pack is a TianshanOS encrypted `.tscfg` package.

2. Quick start for individual users

2.1 Replace default passwords immediately

Use when: setting up a device, responding to password exposure, or changing operators. Who can do it: each signed-in user can use the post-login password prompt for their own password; only root sees Account Security on this page. What changes: the local TianshanOS account password, not any SSH server password.

Change passwords from Account Security

- 1 Sign in as root and open Security.
- 2 In Account Security, choose the root or admin password form.
- 3 Enter the same new password twice and submit.

The UI accepts 4–64 characters. Four characters is only a technical minimum, not a security recommendation. Use a long, unique password.

Expected result: a password-changed message. The current session is not automatically signed out. Verify: sign out or open a private browser window and sign in with the new password. Do not rely on the success message alone. If it fails: correct a mismatch or invalid length and retry. Five consecutive failed logins lock the account for about five minutes. Recovery: root can reset admin to the default password `rm01` and clear the admin lockout. Immediately sign in to admin in a new session and set a unique password.

Do not retain `rm01` as an operating password. The default-password prompt can be dismissed with “Later,” so the device does not force this change for you.

2.2 Create a key for SSH

Use when: the device must connect to a Linux server without sending the account password on every connection. Who sees it in the page: a signed-in admin or root user. This is frontend visibility, not proof of backend role enforcement. Prerequisite: fewer than eight SSH keys are stored. What changes: a new public/private key pair is written to device NVS; no server is changed yet.

- 1 In Key Management, select Generate New Key.
- 2 Enter a unique key ID. A purpose-based name such as `backup-server` is useful; never put a password or secret in the ID.
- 3 Select RSA 2048-bit (recommended), or RSA 4096 if your policy requires it.
- 4 Optionally add a comment and alias.
- 5 Leave Allow Private Key Export off unless you have a defined offline backup or migration requirement.
- 6 Do not treat Hide Key ID as access control. The current implementation can still return the real ID.
- 7 Select Generate.

Why not ECDSA: the page can create ECDSA P-256 and P-384 keys, but the current SSH public-key authentication flow does not support them. The HTTPS ECDSA key belongs to a separate subsystem. Expected result: the new RSA entry appears with Public Key, Deploy, Revoke, and Delete actions. Verify: select Public Key and confirm that the displayed key begins with `ssh-rsa`. If it fails: use another ID if the ID already exists. If storage is full, retire an unused key using the revoke-first sequence in this guide. Recovery: a key that has never been deployed can be deleted directly. Once deployed, revoke it remotely before deleting it locally.

2.3 Deploy the key to a server

Use when: adding the RSA public key to a remote account. Who sees it in the page: a signed-in admin or root; actual deployment also requires the target account password. This is frontend visibility, not proof of backend role enforcement. Prerequisites: the server and port are reachable, the account currently accepts password authentication, and the selected RSA key exists. What changes: the public key is appended to the remote account's `~/.ssh/authorized_keys`; local Deployed Host and Known Host records are also created.

- 1 Select Deploy on the intended key row.
- 2 Enter the target host, username, SSH port, and that remote account's password.
- 3 Recheck the address and account, then select Start Deployment.

Current security boundary: the implementation completes password authentication before it checks the host fingerprint. It also trusts a new host automatically. The first deployment therefore does not provide strict “verify the host before sending the password” protection. Perform first contact on an isolated network and compare the saved fingerprint afterward. Expected result: a deployment message and a new entry under Deployed Hosts. Verify: select Test on that host. The test reconnects with the selected key and runs a fixed echo command. Partial failure: the public key may have been appended even if key-login verification fails. A failure to save the local host record also does not change the successful deployment response, so the new row may be missing. In either case, inspect remote `authorized_keys` through another management path before retrying. Do not repeatedly select Deploy because the current command appends without deduplication. Recovery: use Revoke to remove the remote authorization. Do not delete the local key first.

2.4 Verify the connection correctly

Use when: after deployment, after server maintenance, or during routine checks. Who sees it in the page: a signed-in admin or root. This is frontend visibility, not proof of backend role enforcement. What changes: a fingerprint may be saved on first contact; business configuration is not changed.

- 1 Find the entry under Deployed Hosts.
- 2 Confirm its address, port, username, and key ID.
- 3 Select Test.

What success proves: the device established an SSH session with that key and ran the fixed echo command. It does not prove that sudo, another account, or an application command is permitted. If it fails: check reachability, port, SSH service, account, and whether the key remains in `authorized_keys`. Stop immediately and follow the fingerprint-change process if server identity is involved. Recovery: the test itself needs no recovery. If a wrong new address was trusted, remove that Known Host record and verify the real server identity before reconnecting.

2.5 Revoke access in the correct order

Use when: a device is lost, a private key may be exposed, a server is retired, or the key purpose changes. Who sees it in the page: a signed-in admin or root; actual revocation also requires the remote account password. This is frontend visibility, not proof of backend role enforcement. Prerequisites: another administrative path exists and the local key has not been deleted. What changes: every matching public-key line is removed from the remote `authorized_keys`. A successful revoke initiated from Deployed Hosts also removes the local host record.

- 1 Prefer Revoke on the relevant Deployed Host row.
- 2 Enter the server password and confirm Revoke & Remove.
- 3 Wait for the reported number of removed matches.
- 4 Through another administrative path, inspect `authorized_keys` and prove that the old key can no longer sign in.
- 5 Inspect `~/.ssh/authorized_keys.bak`. The current revoke operation creates this backup, which may still contain the old public key. Handle it according to server backup policy.
- 6 After every target server has been checked, delete the local key.

Key not found: the page offers to remove only the local record. Use that option only after verifying on the server that the key is absent. Revoke failed: retain the local key and record. Use a server console or another

administrator account to remove the public key manually. Recovery: an accidental revoke requires a fresh deployment after server identity is verified. Removing a local record cannot restore remote access.

3. SSH keys and server management

3.1 Key lifecycle

Use this sequence:

- 1 Create an RSA key.
- 2 Save a private-key backup only when policy requires one; non-exportable is safer by default.
- 3 Deploy to a specific server and account.
- 4 Run Test.
- 5 Periodically compare local records with real remote authorization.
- 6 Revoke on every server when retiring the key.
- 7 Prove the key can no longer sign in.
- 8 Delete the local key and stale records last.

3.2 Public and private key export

Export a public key

Purpose: give public material to a server administrator for manual authorization. Impact: no private material is exposed. Steps: select Public Key on the key row, verify the key ID and type, then copy the complete line. Verify: ensure the destination contains one complete, untruncated line. Recovery: none is needed. If the key went to an unintended server, a separate purpose-specific key makes later revocation easier.

Export a private key

Purpose: controlled migration or offline recovery only. Prerequisite: Allow Private Key Export was enabled when the key was created. It cannot be enabled afterward from this page. Impact: the browser receives the entire private key. Clipboard history, downloads, browser context, and backups may create additional copies. Steps: use only a trusted computer on an isolated network, select Private Key, move it immediately to an approved secret store, and remove temporary copies. Verify: confirm it is the intended key and enforce restrictive file permissions. Never paste it into chat, tickets, or logs. Recovery: a private key sent to the wrong party cannot be recalled. Treat it as exposed: revoke it remotely, create and deploy a replacement, then delete the old key.

3.3 Deployed Hosts is not the remote source of truth

This table is a local TianshanOS configuration list. It does not continuously read remote `authorized_keys` files.

- Test success means the key can currently connect and run the test command.
- Test failure may be caused by networking, the SSH service, account state, a fingerprint, or remote authorization.
- Remove deletes only the local record; the remote key may remain active.

- Revoke attempts to change remote authorization and removes the local record only after success.

Host records and Known Host fingerprints are separate. Deleting one does not delete the other.

3.4 SSH host fingerprints

A host fingerprint answers, “Is this still the same SSH server?” The page stores a SHA-256 digest of the host public key as 64 lowercase hexadecimal characters. Many OpenSSH tools display `SHA256:base64`; these strings cannot be compared directly. Ask the administrator for the same representation.

First connection

The current WebUI automatically trusts a new host. After first deployment:

- 1 Select View under Known Host Fingerprints.
- 2 Obtain the server fingerprint through its console, asset system, or another trusted channel.
- 3 Compare the full 64-character value, not the shortened 32-character table display.
- 4 If it differs, revoke any key that may have been deployed and isolate the incident.

Fingerprint changed

A server rebuild, host-key rotation, IP reassignment, or man-in-the-middle attack can cause this.

- 1 Stop retrying. Do not immediately delete the old fingerprint.
- 2 Obtain the new full fingerprint and change reason from a trusted console or administrator.
- 3 Confirm the IP, port, asset identity, and maintenance record.
- 4 Only after confirmation, delete the old fingerprint.
- 5 Reconnect on a trusted network; the current implementation automatically trusts the new fingerprint.
- 6 View and compare the newly stored value.

The source contains a detailed mismatch dialog, but the current Security-page call path does not open it. In practice you may receive only a generic connection error. Do not wait for the “Update Host Key” dialog described by older documentation.

Fingerprint storage risk

Fingerprints are stored in NVS and synchronized to unsigned, plaintext JSON files on the SD card. At startup, the presence of SD-card fingerprint configuration causes the matching NVS data to be cleared and replaced from SD. Treat the SD card as trust data; a saved record is not tamper-proof evidence.

3.5 Import and export SSH host configurations

This is a host-record-specific `.tscfg` workflow, separate from generic Config Pack application.

Export a host configuration

Who can do it: only a Developer device. The page may show Export on a normal device, but the backend rejects the operation. Impact: creates an encrypted package containing address, port, username, authentication type, and key ID. It contains neither the SSH password nor the referenced private key. Steps: on a Developer device, select Export on the host row. Paste the target-device certificate when exporting to another device; otherwise the UI offers self-targeting behavior. Verify: confirm that a `.tscfg` downloaded and verify the target certificate fingerprint over a trusted channel. Recovery: export does not change the record. Delete a package created for the wrong target and export again.

Import a host configuration

Prerequisites: the package targets this device, the SD card is writable, and the receiving device already has the referenced key ID. The package does not transfer private keys. Impact: confirmation saves the `.tscfg` to SD. The host record is not loaded until restart. Steps: select Import Host Configuration, choose the file, review the preview, and enable overwrite only when replacing an intended same-name record. Confirm and restart when instructed. Verify: after restart, compare the address, user, port, and key ID, then select Test. Security warning: “Signature verified” establishes neither signer trust nor that the package targets this device. Recipient matching is checked only when the package is loaded and decrypted after restart. Recovery: use Remove for an incorrect local record and delete the corresponding package from SD. This does not change remote authorization.

4. HTTPS and mTLS

4.1 Three identities to keep separate

- 1 The device private key and device certificate identify the port 443 server to clients.
- 2 A client certificate identifies a computer, service, or person connecting to TianshanOS.
- 3 The CA chain installed on the device validates those client certificates. It does not automatically make a browser trust the device certificate.

The full WebUI remains on HTTP port 80. The separate port 443 service currently exposes health, identity, and permission-test endpoints only. Completing this section does not move the full WebUI to HTTPS.

4.2 Plan names and certificate purposes

Before contacting the CA administrator, decide:

- whether clients use an IP address or DNS name;
- every IP/DNS value required in SAN;
- whether EKU includes server authentication;
- the client-certificate EKU, issuing CA, and role mapping;
- renewal, revocation, and emergency recovery procedures.

The custom CSR form accepts only CN, O, and OU and produces no SAN. If every field is blank, the default CSR path uses the fixed CN `TIANSHAN-DEVICE-001`, includes one current IP SAN when an IP is available, and

includes no DNS SAN. If that does not match the real access address, do not use this form for production issuance. Have the PKI administrator use a controlled process that produces the required names and purposes.

4.3 Generate the HTTPS key

Who sees it in the page: both admin and root see the certificate controls. The generic API does not enforce this role boundary, so network isolation is essential. What changes: a separate ECDSA P-256 private key is generated. Continuing when one exists overwrites it. Steps: select Generate Key under HTTPS Certificate. If an existing-key warning appears, stop and account for the old certificate, Config Packs, and recovery path before overwriting. Expected result: status changes to Key Generated, Awaiting CSR and an `https` key row appears. Verify: refresh the page and confirm that the key remains present. It is not exportable from this UI. Recovery: without a backup of the old private key, overwrite is irreversible. Reissue the device certificate and recreate packages encrypted to the old certificate.

4.4 Generate and issue a CSR

What changes: a request is produced; no certificate is installed and no private key is exposed. Steps: select Generate CSR, enter CN/O/OU or leave all fields blank for the default path, then copy the complete PEM to the CA administrator. Verify: have the CA administrator parse the request and confirm its public key, subject, and SAN. The custom-field path currently has no SAN. If it fails: generate the device key first. Stop using the form if the CA requires DNS/IP SAN and have the PKI administrator create an appropriate request process. Recovery: a CSR can be regenerated. If the private key changes, the old CSR and any certificate issued from it no longer match.

4.5 Install the device certificate

Prerequisite: the PEM certificate must match the current device private key. What changes: the device certificate is saved to certificate storage. Steps: select Install Certificate, paste the complete PEM, and submit. What “installed successfully” means: the current implementation parsed the PEM and matched its public key to the stored private key. It did not perform complete issuer-chain, SAN, EKU, current-validity, or subject-policy validation. Verify: view the certificate and inspect subject, issuer, and validity. Then use the actual target client to validate the chain, address SAN, and server-authentication EKU. Restart the device if needed before testing port 443. If it fails: check PEM boundaries and line breaks. If the key does not match, find the certificate issued from the current CSR rather than blindly replacing the private key. Recovery: install the correct certificate that matches the current key. Deleting all PKI also deletes the key and CA chain, so it is a more disruptive recovery path.

4.6 Install the client CA chain and test mTLS

Use when: clients connecting to port 443 must present certificates. What changes: one or more CA certificates become the trust chain used to validate client certificates. Steps: select Install CA Chain, paste one or more PEM certificates as shown in the UI, and install. Verify: restart the device or the port 443 service, then test with both a correctly issued client certificate and an untrusted or absent certificate. The first should reach only endpoints permitted by its mapped role; the second should fail. Current limitation: static code does not show an automatic reload of a running port 443 service after certificate installation. Confirm restart requirements on the real device. Recovery: keep the HTTP management path available. If an incorrect CA makes port 443 unusable, reinstall the correct CA or rebuild PKI from that management network.

4.7 Delete all PKI credentials

Use when: responding to private-key exposure, retiring the device, or completely rebuilding PKI. Impact: removes the HTTPS private key, device certificate, and client CA chain together. The UI treats this as irreversible. Config Packs requiring the old device private key may also become permanently unreadable. Steps: confirm the HTTP management path, retain necessary public certificates and external recovery material, then select Delete and confirm. Verify: refresh and expect Not Initialized; after restart, prove that port 443 no longer presents the old certificate. Recovery: generate a new key, issue and install a new device certificate, reinstall the client CA chain, and repeat mTLS acceptance tests.

5. Config Pack

5.1 Trust model

A package involves four actions:

- 1 The exporting device encrypts configuration to a target device certificate.
- 2 Only the target holding the corresponding private key can decrypt it.
- 3 The exporting device signs the ciphertext with its device private key.
- 4 The receiver verifies that signature with the signer certificate embedded in the package.

The fourth step currently proves only that the signature matches the certificate carried inside the same package. Signer certificate-chain trust validation is not implemented. The package can provide its own Official field, and a certificate whose OU contains `Developer` is also displayed as official. Therefore:

“Valid signature” does not mean “trusted signer,” and “Official” is not a trustworthy identity conclusion.

Before import, compare the signer certificate or fingerprint with a trusted asset record, controlled offline handoff, or another administrator-approved channel.

5.2 Current capability table

Operation	Actual current result	Production use
Export Device Certificate	Displays/copies a public device certificate for encryption to this device	Usable only with trusted fingerprint verification
Verify pasted package	Verifies structure and the signature against its embedded certificate	Only one part of a trust decision
Security-page Import Config Pack	The frontend sends package content, while the backend currently requires an on-device file path	Not currently functional
Import in Config Pack List	Validates an existing on-device file only; it does not copy, decrypt, or apply it	Does not mean imported or active
Apply	Decrypts and lists top-level module names, but does not call module configuration writers and may still report success	Cannot prove configuration changed
Developer Export Config Pack	Encrypts and signs JSON, offers a download, and attempts to save to SD	Can create a package, but generic receiver application is incomplete

5.3 Export the device certificate

Who sees it in the page: both admin and root see the export entry. This is frontend visibility, not proof of backend role enforcement. Impact: exports a public certificate, never the private key. Steps: select Export Device Certificate, copy the PEM, and record the displayed fingerprint. Verify: have the sender compare that fingerprint through another trusted channel. Do not deliver both certificate and fingerprint solely in the same message. Recovery: sending a public certificate to the wrong recipient normally does not expose the private key, but it may cause packages to target the wrong device. Delete those packages and create replacements.

5.4 Verify a received package

Who sees it in the page: both admin and root see the verification entry. This is frontend visibility, not proof of backend role enforcement. What changes: Verify alone applies no configuration. Steps: open Import Config Pack, choose or paste the `.tscfg`, then select Verify. Record the signer CN, OU, signature time, and Official display. This Verify action does not compare the package recipient fingerprint with this device. Required follow-up: obtain the expected signer certificate fingerprint from a trusted asset record; confirm the target device certificate has not changed; manually review the source, purpose, scope, and rollback plan. Stop when: the signer is unknown, the source cannot be authenticated, the target is wrong, the purpose is unclear, or anyone asks you to ignore a warning.

5.5 Do not use generic Apply as proof of configuration

The current generic Config Pack flow has two independent breaks:

- 1 Upload/paste Import on the Security page sends parameters that the current backend import handler does not accept.
- 2 Even when an on-device package reaches Apply, the implementation only enumerates module names and does not write module configuration, although it may report success.

Therefore:

- Do not treat Imported, Verification Successful, Apply Successful, or the applied-module list as evidence of a configuration change.
- Do not use generic Config Pack for production bulk configuration, disaster recovery, or compliance changes.
- The SSH-host-specific `.tscfg` import is separate. It saves a host record for loading after restart; it carries neither private keys nor remote authorization.

5.6 Developer export

Who can do it: the backend completes export only when the device certificate marks the device as Developer; the page displays the entry to both admin and root. The current `config.admin` value is API-registration metadata, not an enforced account-permission check. Prerequisites: valid JSON configuration files exist on SD and the target-device certificate has been authenticated. Impact: selected JSON files are merged, encrypted to the target certificate, and signed by this device. The UI also attempts to save under `/sdcard/output_config/` and offers copy/download in the browser. Steps: select Export Config Pack, choose one or more JSON files, enter a name and description, paste the target certificate, generate, and download. Verify: independently confirm the browser download and the on-device save path. A failed SD save can still leave package content in the API response. The receiver may verify the package and target fingerprint, but generic application remains incomplete. Recovery: export does not change source configuration. Delete an incorrect package and generate it again. Replacing the target certificate may make old packages undecryptable.

6. Security incidents and recovery

6.1 Suspected SSH private-key exposure

- 1 Restrict device and log access immediately. Current SSH INFO logs may contain the beginning of private-key material.
- 2 Remove the public key from every server through a trusted administrative path; the local host list may be incomplete.
- 3 Inspect and clean `authorized_keys.bak` and other remote backups.
- 4 Prove the old key can no longer sign in.
- 5 Create and deploy a new RSA key.
- 6 Delete the old local key and every exported private-key copy.
- 7 Audit login logs for the exposure window.

6.2 Server rebuild or fingerprint change

Stop connecting → obtain the new fingerprint from the console → confirm the change record and asset identity → delete the old record → reconnect on a trusted network → compare the new saved fingerprint. Stop if any step cannot establish server identity.

6.3 Certificate expiry or HTTPS private-key exposure

- For expiry only, create an appropriate CSR using the current key, install the renewed certificate, and run client validation.
- For private-key exposure, do not reuse the key. Preserve the HTTP management path, replace the PKI set, reissue the device certificate and client-CA configuration, and retire packages encrypted to the old key.

6.4 Remote authorization and local record disagree

- Remote key absent, local record present: prove the key cannot sign in, then Remove the local record.

- Local record absent, remote key present: use the server console or another administrator account to remove the key manually.
- Uncertain: treat the remote `authorized_keys` and a real login test as the source of truth, not the page table.

6.5 Confirm a recovery path before deletion

Before deleting an SSH key, retain a server console or another admin key. Before deleting PKI, retain HTTP management access. Before replacing the device certificate, provide a replacement for any Config Pack that depends on the old certificate. Stop if no recovery path exists.

7. Troubleshooting

Symptom	Check first	Next action	Stop condition
Account Security is missing	Whether you signed in as root	Use root only when account recovery is required	Do not treat normal admin behavior as a fault
Login is locked	Whether five attempts failed	Wait about five minutes; root can reset admin	Stop guessing if identity is uncertain
Generate Key fails	Duplicate ID or eight-key limit	Retire an unused key using the revoke-first order	Stop before deleting a key still in use
ECDSA deployment/test fails	Whether ECDSA was selected for SSH	Create and deploy an RSA key	Do not weaken the server to accommodate the wrong type
Deploy succeeds but Test fails	Address, user, port, remote permissions, algorithm, duplicate lines	Inspect <code>authorized_keys</code> through another path	Stop if the host identity is uncertain
Fingerprint error or unexpected change	Console fingerprint, maintenance record, IP reuse	Follow the fingerprint-change process	Stop immediately if no trusted verification channel exists
Login still works after Remove	Remove changes only the local record	Revoke the key on the server	Do not risk the only admin path
Revoke reports key not found	Correct account/key and possible manual removal	Remove local record only after remote confirmation	Do not claim revocation without remote evidence
Host export fails on normal device	Whether status says Developer	Normal devices cannot currently export it	Do not falsify certificate OU to gain the flag
Client reports name mismatch after issuance	Whether the access address is in SAN	Have PKI issue a correct certificate	Do not rely on CN to bypass SAN validation
Certificate installs but port 443 fails	Key, device certificate, client CA, restart state	Restart, then test trusted and untrusted clients	Stop deleting PKI if no HTTP recovery path exists
Browser still distrusts device	Browser-side server CA trust, SAN, ECU, and chain	Configure the client trust store correctly	Do not confuse device client CA with browser trust
Config Pack Import fails	Whether you used the paste/upload flow	The current parameter contract is broken; stop retrying	Do not fall back to trusting Apply success
Apply reports success but nothing changes	Generic module writers are not implemented	Configure through each supported feature page	Never use this result as production acceptance evidence

8. Security limitations you must account for

Until implementation changes and is reverified, use these boundaries for risk decisions:

- 1 The full WebUI runs over HTTP, so sensitive values can cross the network in plaintext.
- 2 SSH password authentication occurs before host-fingerprint verification, and new hosts are trusted automatically.
- 3 Known Host SD files are unsigned plaintext and take precedence over NVS during startup loading.
- 4 NVS Encryption, Flash Encryption, and Secure Boot are not enabled in the current build; do not claim these protections for private keys.
- 5 Hide Key ID is not currently secrecy or access control.
- 6 SSH logs may contain the beginning of private-key material.
- 7 The generic Web API does not currently enforce a unified login or permission check; `requires_auth` and `permission` are registered but not executed. Certificate installation is also not complete PKI validation.
- 8 The mTLS service on port 443 is not the full WebUI; the CA installed on the device validates client certificates.
- 9 Config Pack does not validate signer certificate-chain trust, and its Official marker is not trustworthy identity evidence.
- 10 Generic Config Pack import and application are incomplete and cannot prove configuration became active.

If your deployment cannot accept any of these limitations, do not place the device inside that trust boundary. Have the security owner choose network isolation, compensating controls, or a product implementation change.